

# Hệ thống máy tính: góc nhìn của lập trình viên

Randal E. Bryant, David R. O'Hallaron

Pearson, ấn bản thứ ba, 2016

*Nguồn gốc: Computer Systems: A Programmer's Perspective, Third Edition*

English Materials / Computer Systems, A Programmer's Perspective 3rd Edition.pdf

## Tóm tắt

Đây là bản ghi chú tiếng Việt mở rộng cho *Computer Systems: A Programmer's Perspective*, ấn bản thứ ba. Sách giải thích hệ thống máy tính từ góc nhìn của người lập trình: biểu diễn dữ liệu, mã máy, kiến trúc xử lý, bộ nhớ, liên kết, luồng điều khiển ngoại lệ, bộ nhớ ảo, I/O, mạng và lập trình đồng thời. Bản này không dịch các mẫu mã hay bài lab thành code mới; nó chuyển ngữ cấu trúc, mục đích học và các khái niệm hệ thống cần nắm để viết chương trình nhanh hơn, đúng hơn và dễ gỡ lỗi hơn.

## 1 Thông tin sách

|                     |                                       |
|---------------------|---------------------------------------|
| <b>Tác giả</b>      | Randal E. Bryant; David R. O'Hallaron |
| <b>Cơ quan</b>      | Carnegie Mellon University            |
| <b>Nhà xuất bản</b> | Pearson                               |
| <b>Ấn bản</b>       | Thứ ba                                |
| <b>Năm</b>          | 2016                                  |
| <b>ISBN</b>         | 978-0-13-409266-9                     |

Sách được xây dựng từ khóa học *Introduction to Computer Systems* tại Carnegie Mellon. Bộ lọc nội dung của sách rất thực dụng: chỉ đi sâu một chủ đề khi nó ảnh hưởng đến hiệu năng, tính đúng đắn hoặc tính hữu dụng của chương trình C ở mức người dùng. Vì vậy sách không phải giáo trình thiết kế phần cứng thuần túy, cũng không phải sách hệ điều hành thuần túy; nó là cầu nối giữa mã nguồn C, trình biên dịch, phần cứng, hệ điều hành và mạng.

## 2 Cách dùng trong thư viện này

Trong bối cảnh OI, không phải mọi chương đều trực tiếp phục vụ thi thuật toán, nhưng nhiều chương giải thích những lỗi và giới hạn mà lập trình viên hiệu năng cao gặp hàng ngày: tràn số, bit-level behavior, stack, cache, locality, linker, memory leak, race và deadlock. Khi dịch sâu hơn, các đoạn code nên được giữ là mã tham khảo của sách; không chuyển chúng thành template mới.

Các thuật ngữ nên dịch nhất quán:

- **cache**: bộ nhớ đệm; **locality**: tính cục bộ.
- **process**: tiến trình; **thread**: luồng.

- **virtual memory**: bộ nhớ ảo; **address space**: không gian địa chỉ.
- **linking**: liên kết; **relocation**: tái định vị.
- **exceptional control flow**: luồng điều khiển ngoại lệ.
- **race**: tranh chấp dữ liệu hoặc race; **deadlock**: bế tắc.

### 3 Cấu trúc lớn của sách

Sách có một chương nhập môn và ba cụm nội dung:

- **Cấu trúc và thực thi chương trình**: Chương 2–6, từ biểu diễn thông tin đến mã máy, kiến trúc xử lý, tối ưu và phân cấp bộ nhớ.
- **Chạy chương trình trên hệ thống**: Chương 7–10, gồm liên kết, tiến trình/tín hiệu, bộ nhớ ảo và I/O mức hệ thống.
- **Tương tác và song song**: Chương 11–12, gồm lập trình mạng và lập trình đồng thời.

Nếu mục tiêu là lập trình thi đấu và hiệu năng, nên đọc Chương 2, 3, 5 và 6 trước. Nếu mục tiêu là hệ thống, bảo mật hoặc backend, đọc tiếp Chương 7–12.

### 4 Ghi chú mở rộng theo chương

#### Chương 1. Một chuyến tham quan hệ thống máy tính

Chương mở đầu theo dấu chương trình hello từ mã nguồn đến khi in ra màn hình. Một chương trình C ban đầu là chuỗi byte có ngữ cảnh; trình biên dịch, assembler và linker lần lượt biến nó thành chương trình thực thi. Khi chạy, processor đọc lệnh từ bộ nhớ, hệ điều hành tạo ảo giác mỗi chương trình có CPU và bộ nhớ riêng, còn I/O được mô hình hóa qua file.

Thông điệp quan trọng nhất là dữ liệu luôn di chuyển: từ disk vào memory, từ memory vào cache, từ cache vào register, rồi ra thiết bị. Vì khoảng cách tốc độ giữa các mức lưu trữ rất lớn, cache và tính cục bộ quyết định hiệu năng thực tế. Sách cũng giới thiệu ba trừu tượng lớn của hệ điều hành: file cho I/O, virtual memory cho memory/disk, và process cho CPU/memory/I/O.

Ba chủ đề xuyên suốt là định luật Amdahl, concurrency/parallelism và trừu tượng hóa. Định luật Amdahl nhắc rằng tăng tốc một phần chương trình chỉ có ích tương ứng với tỷ lệ thời gian phần đó chiếm. Concurrency là nhiều luồng công việc chồng lấn theo thời gian; parallelism là thực thi thật sự đồng thời trên nhiều tài nguyên.

#### Chương 2. Biểu diễn và thao tác thông tin

Chương 2 giải thích vì sao mọi dữ liệu đều là bit cộng ngữ cảnh. Cùng một dãy bit có thể là số nguyên không dấu, số bù hai, ký tự, địa chỉ hoặc lệnh máy tùy cách diễn giải. Các phần đầu trình bày hệ thập lục phân, kích thước kiểu dữ liệu, byte ordering, biểu diễn chuỗi, boolean algebra, phép bit, phép logic và dịch bit trong C.

Phần số nguyên là nền cho nhiều lỗi khó thấy. Số không dấu và số có dấu dùng cùng bit pattern nhưng quy tắc so sánh/chuyển kiểu khác nhau. Tràn số nguyên không phải “số lớn hơn” theo nghĩa toán học; trong biểu diễn hữu hạn, phép cộng, nhân, đổi dấu và cắt bit đều có quy tắc modulo hoặc bù hai cụ thể. Đây là lý do một điều kiện tưởng đúng trong toán có thể sai trong C.

Phần floating point trình bày số nhị phân phân số, chuẩn IEEE, normalized và denormalized values, rounding, infinity và NaN. Bài học cho thuật toán là số thực máy không có tính kết hợp hoàn hảo; thứ tự cộng có thể làm thay đổi sai số, so sánh bằng cần thận trọng, và chuyển giữa int/float có thể mất thông tin.

### **Chương 3. Biểu diễn chương trình ở mức máy**

Chương này đọc mã máy x86-64 như một cách hiểu compiler dịch C. Mục tiêu không phải viết assembly bằng tay, mà là thấy con trỏ, mảng, struct, vòng lặp, switch, lời gọi hàm và đệ quy được hạ xuống register, stack và lệnh nhảy như thế nào.

Các khái niệm chính gồm register, operand, addressing mode, lệnh move, arithmetic/logical operations, condition code, jump, conditional move và switch table. Stack frame giải thích cách truyền tham số, lưu địa chỉ trả về, lưu biến cục bộ và hỗ trợ đệ quy. Những chi tiết này giúp đọc debugger, hiểu backtrace và phát hiện lỗi ghi quá biên.

Phần mảng, con trỏ và struct cho thấy layout bộ nhớ quyết định phép truy cập. Mảng nhiều chiều được tuyến tính hóa; struct có alignment và padding; union chia sẻ cùng vùng nhớ. Phần bảo mật bộ nhớ giải thích buffer overflow và vì sao stack discipline quan trọng. Các đoạn code minh họa trong sách nên giữ nguyên làm ví dụ, không biến thành template.

### **Chương 4. Kiến trúc bộ xử lý**

Chương 4 dùng tập lệnh giáo dục Y86-64 để trình bày cách processor thực thi lệnh. Người đọc đi từ mô hình trạng thái kiến trúc đến logic tổ hợp, logic tuần tự, HCL, rồi thiết kế bộ xử lý tuần tự và pipeline.

Trong thiết kế tuần tự, mỗi lệnh đi qua các pha như fetch, decode, execute, memory, write back và update PC. Pipeline chồng các pha của nhiều lệnh để tăng throughput, nhưng tạo ra hazard: data hazard khi lệnh sau cần kết quả lệnh trước, control hazard khi nhánh chưa biết hướng, và exception khi trạng thái cần được cập nhật chính xác.

Dù không trực tiếp cần cho OI, chương này giúp hiểu vì sao branch prediction, data dependency và instruction-level parallelism ảnh hưởng đến tốc độ. Nó cũng làm rõ rằng “một lệnh” trong C không tương ứng một bước vật lý đơn giản.

### **Chương 5. Tối ưu hiệu năng chương trình**

Chương 5 chuyển từ hiểu máy sang viết chương trình nhanh hơn. Sách phân biệt tối ưu mà compiler có thể làm với tối ưu cần lập trình viên thay đổi cấu trúc mã. Đầu tiên phải đo: thời gian chạy, cycles per element và bottleneck, thay vì đoán bằng cảm giác.

Các kỹ thuật chính gồm loại bỏ lời gọi thừa trong vòng lặp, giảm memory aliasing, dùng biến cục bộ tích lũy, loop unrolling, tăng instruction-level parallelism và giảm critical path. Tối ưu không chỉ là giảm số lệnh; nếu chuỗi phụ thuộc quá dài, processor không thể khai thác song song mức lệnh.

Phần giới hạn bộ nhớ nối sang Chương 6: khi dữ liệu quá lớn hoặc truy cập kém cục bộ, cache miss làm chương trình chậm hơn rất nhiều so với tính toán thuần. Với lập trình thi đấu, bài học thực tế là tránh truy cập ngẫu nhiên không cần thiết, chọn layout dữ liệu phù hợp và tôn trọng locality.

### **Chương 6. Hệ phân cấp bộ nhớ**

Chương 6 mô tả storage technology và memory hierarchy: register, L1/L2/L3 cache, main memory, local disk và remote storage. Mỗi mức nhanh hơn, nhỏ hơn và đắt hơn mức dưới; mức trên đóng vai trò cache cho mức dưới. Tính cục bộ thời gian và không gian là lý do cơ chế này hoạt động.

Cache được mô hình hóa bằng số set  $S$ , associativity  $E$ , block size  $B$  và address bits. Một địa chỉ được tách thành tag, set index và block offset. Cache hit trả dữ liệu nhanh; cache miss phải lấy block từ mức dưới và có thể evict một line cũ. Direct-mapped, set-associative và fully-associative cache là các điểm khác nhau trên trục đơn giản–linh hoạt.

Phần viết chương trình cache-friendly rất quan trọng: duyệt mảng theo thứ tự bộ nhớ, blocking/tiling cho ma trận, giảm working set và tránh conflict miss. Đây là chương có giá trị trực tiếp nhất cho tối ưu hằng số trong thuật toán.

## Chương 7. Liên kết

Liên kết biến nhiều object file thành một executable. Linker giải quyết symbol, ghép section, tái định vị địa chỉ và xử lý thư viện. Người lập trình thường gặp linker qua lỗi “undefined reference”, duplicate symbol, thứ tự thư viện hoặc khác biệt giữa static và dynamic linking.

ELF object file chứa header, section như `.text`, `.rodata`, `.data`, `.bss`, symbol table và relocation entries. Relocation là quá trình sửa các tham chiếu địa chỉ sau khi linker biết section cuối cùng nằm ở đâu. Shared library và position-independent code dùng GOT/PLT để cho phép mã được nạp ở nhiều địa chỉ khác nhau.

Chương cũng giới thiệu library interpositioning, tức chèn một phiên bản hàm để quan sát hoặc thay đổi hành vi gọi thư viện. Đây là kỹ thuật hữu ích trong debug, profiling và sandboxing, nhưng cần dùng cẩn thận.

## Chương 8. Luồng điều khiển ngoại lệ

Exceptional control flow là mọi thay đổi điều khiển không chỉ do nhánh/lỗi gọi hàm thông thường: interrupt, trap, fault, abort, context switch, signal và process control. Hệ điều hành dùng cơ chế này để chia sẻ CPU, xử lý I/O và cung cấp system call.

Process là trừu tượng của chương trình đang chạy. Fork tạo tiến trình con, exec thay ảnh chương trình, wait thu hồi trạng thái, còn shell là ví dụ kết hợp các cơ chế này. Signal là thông báo không đồng bộ gửi đến tiến trình; handler phải được viết theo quy tắc an toàn vì có thể chạy xen vào luồng chính.

Các lỗi hay gặp gồm zombie process do không wait, race giữa signal và main flow, handler gọi hàm không async-signal-safe, và hiểu nhầm thứ tự chạy giữa parent/child. Chương này là nền để viết shell, server và chương trình Unix chắc chắn.

## Chương 9. Bộ nhớ ảo

Bộ nhớ ảo cho mỗi process một không gian địa chỉ riêng, đồng thời cho phép main memory hoạt động như cache của disk. Dịch địa chỉ dùng page table và TLB: virtual address được ánh xạ sang physical address hoặc gây page fault nếu trang chưa có trong memory.

Không gian địa chỉ gồm code, data, heap, shared libraries, stack và vùng kernel. Heap mở rộng/thu hẹp qua allocator như `malloc` và `free`. Dynamic memory allocation đặt ra bài toán throughput, utilization, fragmentation, coalescing, splitting và free-list organization.

Phần lỗi bộ nhớ rất thực tế: dereference con trỏ rác, ghi vượt biên, use-after-free, double free, quên free gây memory leak, đọc vùng chưa khởi tạo và giả định sai về lifetime. Với lập trình C/C++, chương này giải thích nguồn gốc của nhiều lỗi khó debug nhất.

## Chương 10. I/O mức hệ thống

Unix I/O mô hình hóa mọi thứ như file: disk file, terminal, socket và device. Các thao tác lõi là open, close, read, write, lseek và stat. File descriptor là số nguyên nhỏ mà process dùng để tham chiếu file đang mở; redirection chỉ là thay đổi descriptor.

Sách phân biệt I/O cấp thấp với buffered I/O của thư viện chuẩn C. Robust I/O cần xử lý short count, interrupted system call và EOF đúng cách. Metadata và file hierarchy giải thích directory, permission, link và thông tin trạng thái file.

Chương này quan trọng cho server, judge tool, script hệ thống và chương trình xử lý file lớn. Với OI, nó không thay thế stdio/iostream, nhưng giúp hiểu tại sao buffering, descriptor và pipe hoạt động.

## Chương 11. Lập trình mạng

Chương 11 xây dựng mạng từ góc nhìn socket. Client và server giao tiếp bằng endpoint gồm địa chỉ IP và port. Các hàm hiện đại như getaddrinfo và getnameinfo hỗ trợ lập trình độc lập giao thức và an toàn hơn các hàm cũ.

Mô hình client-server được minh họa bằng web server nhỏ. HTTP request cho nội dung tĩnh trả file từ disk; request cho nội dung động chạy chương trình con theo quy ước CGI rồi trả output. Các khái niệm như byte ordering, connection, DNS, MIME type và proxy được nối với code C hệ thống.

Với người học thuật toán, chương này ít liên quan trực tiếp đến contest cổ điển, nhưng hữu ích khi xây tool, judge nội bộ, service hoặc hiểu latency/throughput trong hệ phân tán.

## Chương 12. Lập trình đồng thời

Chương cuối trình bày concurrent programming bằng process, I/O multiplexing và thread. Thread chia sẻ không gian địa chỉ nên nhẹ hơn process, nhưng đồng thời mở ra race trên dữ liệu dùng chung. Semaphore, mutex và condition variable là các công cụ đồng bộ chính.

Server đồng thời cần xử lý nhiều client mà không chặn toàn bộ tiến trình. Mỗi mô hình có tradeoff: process tách biệt tốt nhưng nặng, event-driven tiết kiệm nhưng khó viết, thread tự nhiên hơn nhưng cần khóa đúng. Chương cũng bàn đến thread-level parallelism trên multicore.

Các lỗi cốt lõi là race, deadlock, livelock, starvation và không giữ invariant khi nhiều luồng xen kẽ. Với code hiệu năng cao, đúng về mặt thuật toán chưa đủ; cần đúng dưới mọi lịch chạy hợp lệ.

## 5 Phụ lục kỹ thuật cho lập trình thi đấu và hệ thống

Phần này bổ sung các điểm trong CS:APP thường tác động trực tiếp đến chương trình C/C++ khi chạy trên judge, benchmark hoặc môi trường Unix. Mục tiêu là dịch thành tri thức kiểm tra được: mỗi mục nêu nguyên nhân lỗi, dấu hiệu và cách đọc lại theo mô hình hệ thống của sách.

### 5.1 Bit, kiểu số và chuyển kiểu

Một dãy bit không tự mang nghĩa. Cùng  $w$  bit có thể được đọc thành số không dấu trong khoảng  $0, \dots, 2^w - 1$ , hoặc số bù hai trong khoảng  $-2^{w-1}, \dots, 2^{w-1} - 1$ . Vì vậy phép so sánh và mở rộng kiểu trong C/C++ có thể đổi kết quả khi trộn signed và unsigned.

Các lỗi cần kiểm tra:

- so sánh int âm với size\_t làm về âm bị chuyển thành số không dấu rất lớn;

- lấy trị tuyệt đối của giá trị nhỏ nhất kiểu signed bị tràn vì không có số dương tương ứng;
- nhân hai số 32-bit rồi mới gán vào 64-bit vẫn có thể tràn trước khi gán;
- dịch trái số signed âm hoặc dịch quá độ rộng kiểu là hành vi không xác định theo C/C++.

Khi công thức toán cần modulo  $2^w$ , dùng kiểu unsigned để biểu diễn rõ ý định. Khi công thức cần số nguyên toán học, ép toán hạng lên kiểu đủ rộng trước khi cộng/nhân, ví dụ dùng `long long` hoặc `__int128` cho tích tạm thời trong hình học và số học modulo.

## 5.2 Số thực IEEE và sai số

Floating point có ba lớp đặc biệt: số chuẩn hóa, số không chuẩn hóa, và các giá trị ngoại lệ như  $\pm\infty$ , NaN. Phép cộng/nhân làm tròn về số biểu diễn được gần nhất, nên các luật đại số quen thuộc không luôn đúng:

$$(a + b) + c \neq a + (b + c)$$

có thể xảy ra vì sai số làm tròn và mất chữ số có nghĩa.

Với bài thuật toán:

- so sánh bằng số thực nên thay bằng ngưỡng sai số khi đề cho phép;
- với hình học tọa độ nguyên, ưu tiên tích có hướng bằng số nguyên thay vì lượng giác hoặc căn bậc hai;
- khi cộng nhiều số có độ lớn rất khác nhau, cộng từ nhỏ đến lớn hoặc dùng kỹ thuật bù sai số nếu cần độ chính xác cao;
- NaN không bằng chính nó, nên một phép tính lỗi có thể làm điều kiện rẽ nhánh hoạt động khác dự đoán.

## 5.3 Đọc assembly để gỡ lỗi

CS:APP dùng assembly như ngôn ngữ quan sát compiler. Khi chương trình crash, backtrace và disassembly giúp trả lời ba câu hỏi: lệnh đang chạy là gì, register nào chứa địa chỉ hoặc dữ liệu lỗi, và stack frame hiện tại được dựng ra sao.

Các dấu hiệu thường gặp:

- địa chỉ gần 0 thường là dereference con trỏ null hoặc offset từ null;
- địa chỉ có mẫu lạ sau khi `free` gợi ý use-after-free;
- stack pointer lệch hoặc địa chỉ trả về bị ghi đè gợi ý buffer overflow;
- vòng lặp bị tối ưu làm mất biến local trong debugger nếu build với tối ưu cao.

Khi cần debug sâu, build bản riêng với `-g`, tắt bớt tối ưu nếu cần, và so sánh với bản tối ưu để tránh kết luận sai do thay đổi hành vi undefined. Mục tiêu không phải viết assembly, mà là hiểu phép truy cập mảng, struct, con trỏ và lời gọi hàm đã được hạ xuống địa chỉ thật như thế nào.

## 5.4 Cache, locality và layout dữ liệu

Mô hình cache trong Chương 6 giải thích vì sao hai thuật toán cùng  $O(n)$  có thể chênh lệch rất lớn. Nếu block size là  $B$ , một cache miss thường đưa cả một đoạn byte liên tiếp vào cache. Duyệt mảng tuyến tính tận dụng spatial locality; lặp lại trên cùng dữ liệu trong thời gian ngắn tận dụng temporal locality.

Các quy tắc thực tế:

- duyệt mảng hai chiều theo đúng thứ tự lưu trong bộ nhớ; C/C++ lưu row-major, nên vòng trong thường là cột cuối cùng;
- gom dữ liệu hay truy cập cùng nhau vào cấu trúc liền kề, tránh con trỏ nhảy ngẫu nhiên nếu không cần;
- dùng blocking/tiling cho phép nhân ma trận, DP hai chiều lớn hoặc xử lý ảnh/lưới để working set nằm trong cache;
- tách dữ liệu nóng và lạnh khi một struct lớn chỉ cần vài trường trong vòng lặp chính;
- tránh false sharing trong code đa luồng khi nhiều luồng ghi các biến khác nhau nhưng nằm cùng cache line.

Trong lập trình thi đấu, tối ưu cache không thay thế thuật toán đúng, nhưng nó có thể quyết định giữa AC và TLE khi giới hạn sát. Trước khi tối ưu vi mô, cần đo bằng input đại diện và kiểm tra bottleneck có thật nằm ở bộ nhớ hay không.

## 5.5 Compiler, aliasing và tối ưu

Compiler chỉ tối ưu trong phạm vi hành vi được ngôn ngữ bảo đảm. Nếu chương trình có undefined behavior, compiler được phép giả định trường hợp đó không xảy ra và sinh mã làm kết quả nhìn như “phi lý”. Các nguồn UB phổ biến là tràn signed integer, truy cập ngoài mảng, dùng biến chưa khởi tạo, vi phạm strict aliasing và dịch bit sai.

Aliasing làm compiler thận trọng: nếu hai con trỏ có thể trỏ vào cùng vùng nhớ, một phép ghi qua con trỏ này có thể làm hỏng giá trị đọc qua con trỏ kia. Vì vậy biến cục bộ tích lũy, mảng riêng biệt và giao diện rõ ownership thường giúp compiler tối ưu tốt hơn. Với C, restrict biểu thị cam kết không alias; với C++, cần thận trọng hơn và thường dựa vào cấu trúc dữ liệu rõ ràng.

## 5.6 Linking và lỗi biên dịch nhiều file

Nhiều lỗi “biên dịch được file đơn, nhưng build toàn bộ thất bại” đến từ linking. Linker cần một định nghĩa duy nhất cho mỗi symbol mạnh, giải quyết mọi tham chiếu chưa biết, và đặt section vào địa chỉ cuối cùng. Header chỉ nên chứa khai báo, inline/template hợp lệ, hoặc định nghĩa có liên kết nội bộ; biến toàn cục định nghĩa trong header dễ tạo duplicate symbol.

Khi gặp lỗi linking:

- undefined reference: có khai báo nhưng không có object/library chứa định nghĩa, hoặc thứ tự thư viện sai;
- multiple definition: cùng symbol được định nghĩa ở nhiều translation unit;
- ABI mismatch: object được build bằng cấu hình, chuẩn C++ hoặc compiler không tương thích;
- thiếu runtime/shared library khi chạy, dù link đã thành công.

Trong repo dịch thuật này, bài học tương ứng là không biến code mẫu của sách thành template mới nếu không kiểm thử build đầy đủ; code minh họa nên giữ vai trò minh họa.

## 5.7 Bộ nhớ ảo, allocator và lỗi heap

Bộ nhớ ảo tạo ảo giác địa chỉ liên tục, nhưng allocator phải quản lý các block trên heap thật. Một allocator tốt cân bằng throughput và utilization; một chương trình dùng allocator sai có thể leak, fragment hoặc crash.

Các lỗi C/C++ cần đọc theo lifetime:

- **use-after-free**: dùng con trỏ sau khi vùng nhớ đã trả lại;
- **double free**: trả cùng block hai lần, làm hỏng metadata heap;
- **memory leak**: mất mọi con trỏ tới block còn sống;
- **out-of-bounds**: ghi qua cuối block, có thể làm hỏng block kế bên;
- **dangling reference**: tham chiếu tới biến local đã rời scope.

Với C++, `RAII`, `vector`, `string`, `unique_ptr` và `shared_ptr` giúp biểu diễn ownership tốt hơn, nhưng không loại bỏ lỗi logic. Iterator/reference của container có thể bị vô hiệu sau `resize`, `erase` hoặc `rehash`; đây là cùng vấn đề lifetime dưới dạng thư viện chuẩn.

## 5.8 Unix I/O và buffering

Unix xem file, terminal, pipe và socket qua file descriptor. `read` và `write` có thể xử lý ít byte hơn yêu cầu; code chắc chắn phải lặp cho đến khi đủ dữ liệu, gặp EOF hoặc gặp lỗi thật. Lỗi bị ngắt bởi signal `EINTR` có thể cần gọi lại.

Buffered I/O của `stdio` và `iostream` thêm bộ đệm ở user space. Trộn nhiều tầng I/O trên cùng descriptor mà không flush/sync có thể gây thứ tự khó đoán. Trong thi đấu, tắt đồng bộ C++ bằng `ios::sync_with_stdio(false)` chỉ an toàn khi không trộn với `scanf/printf` trên cùng luồng.

## 5.9 Process, signal và thứ tự thực thi

`fork` tạo tiến trình con bằng cách sao chép trạng thái logic của tiến trình cha; sau `fork`, cả hai tiến trình tiếp tục chạy từ điểm trả về. Thứ tự chạy giữa cha và con không được bảo đảm. Nếu cha không `wait`, tiến trình con đã kết thúc có thể thành zombie cho đến khi được thu hồi.

Signal là điều khiển bất đồng bộ, vì vậy handler phải rất nhỏ và chỉ gọi các hàm `async-signal-safe`. Mẫu an toàn là handler đặt cờ kiểu `volatile sig_atomic_t`, còn luồng chính kiểm tra cờ và xử lý đầy đủ. Không nên giữ invariant phức tạp chỉ bằng giả định signal sẽ đến ở thời điểm “thuận tiện”.

## 5.10 Concurrency

Trong chương trình đa luồng, race xảy ra khi hai luồng truy cập cùng dữ liệu, ít nhất một truy cập là ghi, và thiếu đồng bộ hóa tạo quan hệ `happens-before`. Kết quả có thể phụ thuộc lịch chạy, nên test nhiều lần vẫn không chứng minh đúng.

Checklist đồng bộ:

- xác định dữ liệu chia sẻ và lock bảo vệ từng invariant;
- giữ thứ tự lấy lock nhất quán để tránh deadlock;
- dùng condition variable trong vòng `while`, không dùng `if`, vì có thể thức dậy giả hoặc điều kiện đã đổi;



- không giữ lock khi gọi code không kiểm soát hoặc thao tác I/O chậm nếu có thể tránh;
- đo contention, vì thêm luồng không tự động tăng tốc nếu bottleneck là lock hoặc bộ nhớ.

CS:APP chủ yếu dạy cách nhìn hệ thống: đúng đắn phải xét mọi xen kẽ hợp lệ của các luồng, còn hiệu năng phải xét cả CPU, cache, bộ nhớ và kernel, không chỉ số lệnh trong mã nguồn.

## 6 Bài lab và giá trị thực hành

Sách đi kèm các lab nổi tiếng:

- **Data Lab:** hiểu bit-level representation bằng các hàm C bị giới hạn phép toán.
- **Binary Bomb Lab:** đọc assembly và dùng debugger để suy luận input.
- **Buffer Overflow Lab:** hiểu stack và lỗi tràn bộ đệm.
- **Architecture/Performance/Cache Lab:** quan sát processor, tối ưu và cache qua thực nghiệm.
- **Shell/Malloc/Proxy Lab:** viết thành phần hệ thống thật ở mức nhỏ.

Những lab này là lý do sách mạnh về thực hành. Trong bản dịch, nên giữ tên lab, mô tả mục tiêu và cảnh báo an toàn; không biến lời giải hoặc code mẫu thành template.

## 7 Ghi chú về trích xuất

Tệp PDF có 1120 trang và được xử lý qua OCR của Acrobat. Văn bản tiếng Anh trích xuất được nhưng có lỗi nhận dạng ở tiêu đề, dấu câu, chữ trong hình và một số ký hiệu. Vì sách chứa nhiều đoạn mã, hình, bảng và layout hai cột, bản dịch đầy đủ cần đổi chiều trang gốc khi chuyển LaTeX; code nên giữ nguyên và chỉ dịch phần giải thích xung quanh.

## 8 Tình trạng bản dịch

Bản hiện tại là ghi chú dịch mở rộng và hướng dẫn chương, không phải bản dịch từng trang của sách. So với bản ghi chú ban đầu, nó đã bổ sung góc nhìn của khóa học, cấu trúc lớn, thuật ngữ thống nhất, nội dung kỹ thuật của 12 chương, và vai trò của các lab hệ thống.